

Multi-Class Sentiment Classification of Airline Tweets

Using TF-IDF and Machine Learning

Task 2 Report | Artificial Intelligence Internship, Progree

Prepared by: Muhammad Moez Manzoor

Date: September 2026

Source code: <https://github.com/moezmanzoor2003-creator/nlp-sentiment-classifier>

1. Introduction

Sentiment analysis is a Natural Language Processing (NLP) task in which a system decides what opinion is expressed in a piece of text. Airlines receive thousands of customer tweets every day. Sorting them automatically into positive, negative and neutral groups helps a company react quickly to complaints and measure customer satisfaction.

This report describes the design, implementation and evaluation of a multi-class text classifier built in Python with Scikit-Learn. The pipeline cleans raw tweets, removes stop-words, applies lemmatization, converts the text into numerical features with TF-IDF, trains three classifiers and evaluates them using the F1-score.

The objectives of this task were to:

- build a text classification model in Python using Scikit-Learn;
- preprocess the text by removing stop-words and applying lemmatization;
- convert text into numerical data using TF-IDF; and
- evaluate the model with precision, recall and F1-score.

2. Dataset

The Twitter US Airline Sentiment dataset (Crowdfunder / Figure Eight, available on Kaggle) contains 14,640 tweets about major US airlines collected in February 2015. Each tweet is labelled as positive, negative or neutral. Only two columns were used: the tweet text and the sentiment label (airline_sentiment).

Class	Number of tweets	Share of dataset
Negative	9,178	62.7%
Neutral	3,099	21.2%
Positive	2,363	16.1%
Total	14,640	100%

Table 1: Class distribution of the dataset

The dataset is imbalanced. Negative tweets form almost two thirds of the data, while positive tweets are only about 16%. Because of this, accuracy alone can be misleading: a model that always predicts "negative" would already reach roughly 63% accuracy without learning anything useful. For this reason, macro F1-score was chosen as the main evaluation metric.

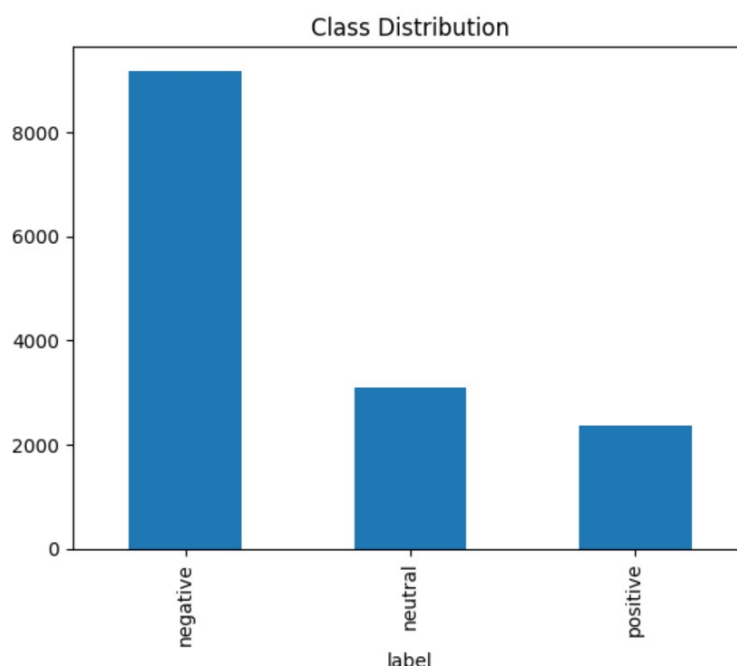


Figure 1: Class distribution of the dataset

3. Methodology

3.1 Text preprocessing

Raw tweets contain links, user mentions, symbols and very common words that carry no sentiment. Each tweet was passed through the following cleaning steps:

- **Lowercasing:** so that "Great" and "great" are treated as the same word.
- **Noise removal:** URLs, @mentions, numbers, punctuation and emojis were removed with regular expressions.
- **Contraction handling:** "n't" was expanded to "not" (for example "don't" becomes "do not").
- **Stop-word removal:** common English words such as "the", "is" and "and" were removed using the NLTK stop-word list. Single-character tokens were also dropped.
- **Lemmatization:** words were reduced to their dictionary form using the WordNet lemmatizer (for example "flights" becomes "flight" and "running" becomes "run"). Lemmatization was applied first as noun and then as verb.

An important design decision was to **keep negation words** ("no", "not", "nor", "never", "against") in the text. The default NLTK stop-word list contains them, but removing them would turn "not good" into "good" and reverse the sentiment.

Original tweet	After preprocessing
@united I don't like the flight delays!! http://t.co/abc	not like flight delay

Table 2: Example of preprocessing

	text	clean
2	@VirginAmerica I didn't today... Must mean I n...	not today must mean need take another trip
3	@VirginAmerica it's really aggressive to blast...	really aggressive blast obnoxious entertainmen...
4	@VirginAmerica and it's a really big bad thing...	really big bad thing
5	@VirginAmerica seriously would pay \$30 a fligh...	seriously would pay flight seat not play reall...
6	@VirginAmerica yes, nearly every time I fly VX...	yes nearly every time fly vx ear worm go away
7	@VirginAmerica Really missed a prime opportuni...	really miss prime opportunity men without hat ...
8	@virginamerica Well, I didn't...but NOW I DO! :-D	well not
9	@VirginAmerica it was amazing, and arrived an ...	amaze arrive hour early good

Figure 2: Tweets before and after preprocessing

3.2 Feature extraction with TF-IDF

Machine learning models work with numbers, not text, so the cleaned tweets were converted into vectors using TF-IDF (Term Frequency, Inverse Document Frequency). The TF-IDF weight of a term t in a document d is the product of how often the term appears in that document (TF) and how rare it is across all documents (IDF):

$$tf-idf(t, d) = tf(t, d) \times \log(N / df(t))$$

where N is the number of documents and $df(t)$ is the number of documents containing the term (Scikit-Learn uses a smoothed version of this formula). Words that appear in almost every tweet, such as "flight", receive a low weight, while less frequent but informative words such as "terrible" receive a higher weight. The vocabulary was limited to the 5,000 most important terms (`max_features = 5000`).

3.3 Train/test split

The data was split into 80% training (11,712 tweets) and 20% testing (2,928 tweets) using a stratified split with `random_state = 42`. Stratification keeps the class proportions the same in both parts. The TF-IDF vectorizer was fitted on the training data only and then applied to the test data, so that no information from the test set leaked into training.

3.4 Classification models

Three classifiers were trained on the same TF-IDF features and compared:

- **Logistic Regression** (`max_iter = 1000`, `class_weight = "balanced"`): a strong and fast baseline for text classification.
- **Linear SVM** (`LinearSVC`, `class_weight = "balanced"`): finds the separating boundary with the largest margin and often performs very well on TF-IDF features.
- **Multinomial Naive Bayes** (default settings): a simple probabilistic model that works on word counts and weights.

Setting `class_weight` to "balanced" gives more importance to the smaller classes and helps against the class imbalance described in Section 2.

4. Evaluation Metrics

- **Precision:** of all tweets the model assigned to a class, the share that truly belong to it.
- **Recall:** of all tweets that truly belong to a class, the share the model found.
- **F1-score:** the harmonic mean of the two, $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.
- **Macro F1:** the unweighted average of the F1 of all classes; it treats every class equally and is the fairest metric for imbalanced data.
- **Weighted F1:** the average of the class F1-scores weighted by class size.

5. Results

5.1 Logistic Regression (baseline)

Table 3 shows the classification report of the Logistic Regression model on the 2,928 test tweets.

Class	Precision	Recall	F1-score	Support
Negative	0.8969	0.8011	0.8463	1835
Neutral	0.5381	0.6952	0.6066	620
Positive	0.6680	0.6892	0.6785	473
Accuracy			0.7606	2928
Macro average	0.7010	0.7285	0.7105	2928
Weighted average	0.7839	0.7606	0.7684	2928

Table 3: Classification report of Logistic Regression

The model reached an accuracy of 76.06%, a **macro F1-score of 0.7105** and a weighted F1-score of 0.7684.

5.2 Model comparison

Model	Macro F1-score
Logistic Regression	0.7105
Linear SVM	0.7053
Multinomial Naive Bayes	0.5313

Table 4: Macro F1-score of the three models

The best model was **Logistic Regression** with a macro F1-score of **0.7105**. Linear SVM came very close (0.7053), while Multinomial Naive Bayes was clearly weaker (0.5313). Naive Bayes was the only model trained without class weighting, which likely made it favour the majority (negative) class and lowered its macro F1-score.

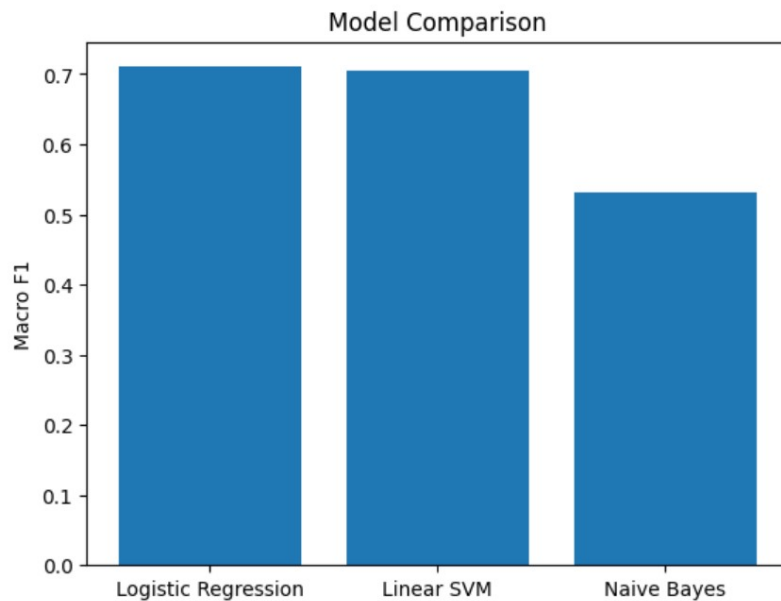


Figure 3: Macro F1-score of the three models

5.3 Confusion matrix

Figure 4 shows the confusion matrix of the Linear SVM model, the second-best model, whose macro F1-score (0.7053) is very close to that of Logistic Regression. Rows show the true class and columns show the predicted class.

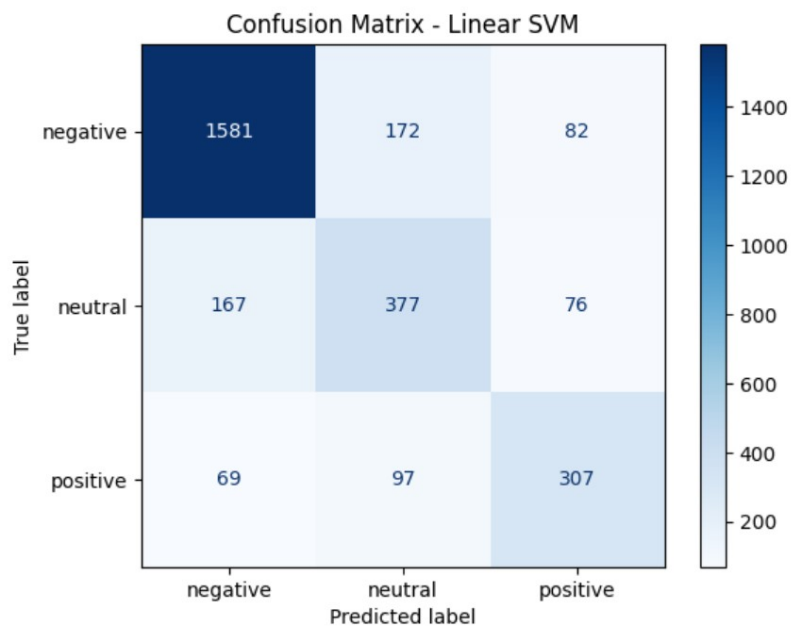


Figure 4: Confusion matrix of the Linear SVM model on the test set

Of the 2,928 test tweets, 2,265 were classified correctly (1,581 negative, 377 neutral and 307 positive), which is about 77.4%. Most errors involve the neutral class: 172 negative tweets were predicted as neutral, 167 neutral tweets were predicted as negative and 97 positive tweets were predicted as neutral. Only 377 of the 620 neutral tweets (60.8%) were identified correctly, the lowest recall of the three classes. Direct confusion between positive and negative is the smallest (82 and 69 tweets), which

shows that the model separates opposite sentiments well and mainly struggles with tweets whose sentiment is weak.

5.4 Predictions on new sentences

The saved Linear SVM model was also tested on sentences it had never seen. The vectorizer and the model were saved together with joblib so they can be reused without retraining.

Input sentence	Intended label	Model output
The crew was amazing and very kind	positive	positive
This is not good, I waited for hours	negative	negative
My flight is at 5 pm tomorrow	neutral	neutral

Table 5: Predictions on new sentences

6. Discussion

The Logistic Regression results show a clear difference between classes. The negative class has the highest F1-score (0.8463) with a precision of 0.8969, which is expected because it is by far the largest class and contains many strong sentiment words such as "delay", "worst" and "cancelled". The positive class reaches 0.6785, with precision and recall almost equal, so the model is balanced on this class even though it is the smallest.

The neutral class is the weakest, with an F1-score of 0.6066. Its recall (0.6952) is clearly higher than its precision (0.5381), which means the model finds most neutral tweets but also labels many non-neutral tweets as neutral. This is likely because neutral tweets, such as questions or factual statements, contain few sentiment words, so their vocabulary overlaps strongly with the other two classes. The confusion matrix in Section 5.3 supports this: most errors involve the neutral class, while positive and negative tweets are rarely mistaken for each other.

The gap between accuracy (0.7606) and macro F1 (0.7105) confirms the effect of class imbalance: the weaker performance on the two smaller classes pulls the macro average down. This is why macro F1 was used as the main metric instead of accuracy.

Keeping negation words during stop-word removal was important for correct results, as shown by the sentence "This is not good, I waited for hours", which the model classified correctly as negative.

7. Conclusion and Future Work

A complete multi-class sentiment classification pipeline was built with Python, NLTK and Scikit-Learn. Text was cleaned, stop-words were removed, words were lemmatized and tweets were converted to TF-IDF vectors. The best result was a macro F1-score of 0.7105 for Logistic Regression, followed closely by Linear SVM (0.7053), evaluated on 2,928 unseen tweets. Multinomial Naive Bayes reached only 0.5313.

The following improvements could raise performance further:

- adding bigrams (`ngram_range = (1, 2)`) so that phrases such as "not good" become single features;
- tuning hyperparameters with `GridSearchCV` and cross-validation;

- using pre-trained embeddings such as Sentence-BERT, or fine-tuning a transformer model like BERT;
- handling class imbalance with resampling techniques and collecting more neutral examples;
- carrying out a detailed error analysis on the misclassified tweets.

8. Tools and References

Tools: Python, Pandas, NLTK, Scikit-Learn, Matplotlib, Joblib, Google Colab.

- Crowdfunder / Figure Eight. Twitter US Airline Sentiment dataset. Kaggle.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.